



Figura 1: Logótipo Ubi.

Relatório de POO

Alunos:

Sofia Costa N°49790
Alexandre Machado N°51882
João Freire N°52216
Hugo Serra N°52297

27 de Dezembro de 2025

Agradecimentos

Gostaríamos de expressar a nossa gratidão à Professora Maria Paula Prata de Sousa, orientadora da Unidade Curricular de Programação Orientada a Objetos, pela sua constante disponibilidade, orientação pedagógica e incentivo ao longo do semestre. O seu compromisso com o ensino e a clareza na transmissão de conhecimentos foram fundamentais para a realização dos objetivos deste trabalho. Agradecemos também os materiais didáticos e o ambiente propício à aprendizagem, que nos ajudaram a desenvolver competências essenciais e a superar os desafios do projeto.

Resumo

Este relatório descreve uma aplicação desktop desenvolvida em *JavaFX* para gestão de hobbies pessoais. A solução permite autenticação simples, criação e manutenção de hobbies, registo de sessões de prática, visualização de estatísticas e personalização de preferências de interface, com persistência local em disco. É apresentada a estrutura da solução por camadas, as funcionalidades implementadas e a justificação das opções técnicas adotadas, bem como limitações e propostas de melhoria.

Conteúdo

0.1	Introdução e objetivo	3
0.2	Visão geral da aplicação	3
0.3	Funcionalidades implementadas	3
0.3.1	Autenticação e utilizadores	3
0.3.2	Gestão de hobbies	3
0.3.3	Gestão de sessões	3
0.3.4	Estatísticas e visualização	4
0.3.5	Preferências e persistência	4
0.4	Estrutura da solução e organização em camadas	4
0.4.1	Camada de interface (ui/)	4
0.4.2	Camada de serviços (services/)	4
0.4.3	Modelos de domínio (models/)	5
0.4.4	Persistência (data/)	5
0.4.5	Estrutura de pastas (resumo)	5
0.5	Fluxos principais da aplicação	5
0.5.1	Arranque	5
0.5.2	Registo e login	5
0.5.3	Gestão de hobbies	5
0.5.4	Gestão de sessões	6
0.5.5	Estatísticas e definições	6
0.6	Modelo de dados	6
0.6.1	Entidade User	6
0.6.2	Entidade Hobby	6
0.6.3	Entidade Sessão	6
0.6.4	Estrutura agregadora AppData	6
0.7	Persistência e salvaguarda	6
0.7.1	Dados principais	6
0.7.2	Preferências globais (pré-login)	7
0.7.3	Momento de gravação	7
0.8	Interface e experiência de utilização	7
0.8.1	Estrutura de navegação	7
0.8.2	Temas e estilos	7
0.8.3	Validações e <i>feedback</i>	7
0.8.4	Gráficos	7
0.9	Justificação das opções técnicas	7
0.9.1	JavaFX + FXML + CSS	7
0.9.2	Serialização local (<i>ObjectOutputStream</i>)	8
0.9.3	AppState como ponto central	8
0.9.4	Preferências aplicadas antes do login	8
0.9.5	Hobbies sugeridos via enumeração	8
0.10	Limitações e melhorias futuras	8

0.11	Como executar e testar	9
0.11.1	Requisitos	9
0.11.2	Execução	9
0.11.3	Testes manuais sugeridos	9
0.12	Conclusão	9

0.1 Introdução e objetivo

O objetivo do projeto é disponibilizar uma aplicação desktop que suporte a gestão organizada de hobbies pessoais, permitindo registar sessões de prática e obter uma visão rápida de estatísticas relevantes (por exemplo, total de sessões, tempo acumulado e hobby mais frequente). A aplicação foi desenhada para funcionar localmente (cenário *offline*), com persistência de dados em disco e preferências de visualização por utilizador.

0.2 Visão geral da aplicação

A aplicação disponibiliza:

- **Autenticação simples:** ecrãs de *login* e registo com validação mínima.
- **Gestão de hobbies:** operações de criar, editar, apagar e filtrar; suporte a sugestões pré-definidas.
- **Registo de sessões:** associação de sessões a hobbies com data/hora e cálculo automático de duração.
- **Pesquisa e filtragem:** pesquisa textual e filtros rápidos nas tabelas.
- **Estatísticas:** cartões de resumo e gráficos de barras (sessões e tempo por hobby).
- **Preferências:** modo escuro, formatos de hora/data, cor dos gráficos e ícone da aplicação.
- **Persistência local:** dados de utilizadores e preferências guardados em disco.

0.3 Funcionalidades implementadas

Nesta secção apresentam-se as funcionalidades efetivamente implementadas, organizadas por áreas.

0.3.1 Autenticação e utilizadores

1. **Registo de conta:** criação de utilizador com deteção de nome de utilizador existente.
2. **Login:** validação de credenciais e carregamento do estado do utilizador autenticado.
3. **Validação mínima:** verificação básica de *inputs* (campos vazios e coerência).

0.3.2 Gestão de hobbies

1. **CRUD completo:** criar, editar e apagar hobbies.
2. **Categorias:** utilização de *Categoria* para classificar hobbies.
3. **Sugestões:** suporte a hobbies sugeridos via *PredefinedHobbies*.
4. **Pesquisa:** filtro por nome/descrição/categoria com atualização imediata da tabela.

0.3.3 Gestão de sessões

1. **CRUD completo:** criar, editar e apagar sessões associadas a um hobby.
2. **Validações:** consistência de data/hora (e.g., início antes do fim).
3. **Cálculo automático:** duração calculada em minutos, com base no intervalo temporal.
4. **Pesquisa:** filtro textual na listagem de sessões.

0.3.4 Estatísticas e visualização

1. **Resumo:** totais de sessões e minutos, duração média e hobby mais frequente.
2. **Gráficos:** dois gráficos de barras (sessões por hobby e tempo por hobby).
3. **Recentes:** tabela/lista de sessões recentes para consulta rápida.

0.3.5 Preferências e persistência

1. **Preferências por utilizador:** tema, formato 12/24h, formato de data e cor dos gráficos.
2. **Preferências globais (pré-login):** tema/formatos aplicados já no ecrã inicial.
3. **Persistência automática:** gravação após alterações em dados e preferências.

0.4 Estrutura da solução e organização em camadas

A solução foi estruturada por camadas, com separação clara entre interface, lógica de aplicação, modelos de domínio e persistência de dados.

0.4.1 Camada de interface (ui/)

A camada ui/ contém:

- **Arranque da aplicação:** classe `App`, responsável por iniciar o JavaFX, aplicar o tema e carregar vistas.
- **Vistas FXML:** definem a estrutura visual (layouts e componentes).
- **Controladores:** tratam eventos, validação e navegação entre ecrãs.
- **CSS:** estilos base e modo escuro.

Controladores principais (exemplos):

- `LoginController` e `RegisterController`: autenticação e criação de conta.
- `MainController`: navegação e carregamento de sub-vistas.
- `HobbiesController` e `SessionsController`: gestão de hobbies e sessões.
- `StatsController`: cálculo e apresentação de estatísticas e gráficos.
- `SettingsController`: alteração de preferências e operações de manutenção (e.g., limpeza de dados).

0.4.2 Camada de serviços (services/)

A camada `services/` centraliza a lógica de aplicação e o estado:

- **`AppState`:** ponto único de verdade para o utilizador autenticado, estado global e persistência.
- **`GestorHobbies`:** operações sobre hobbies e apoio à lógica de negócio.
- **`PredefinedHobbies`:** disponibiliza sugestões para acelerar o *onboarding*.

0.4.3 Modelos de domínio (`models/`)

A camada `models/` contém as entidades serializáveis:

- **User:** credenciais, listas de hobbies e sessões, e preferências.
- **Hobby:** nome, categoria e descrição.
- **Sessão:** hobby associado, data, hora de início/fim (ou duração) e notas.
- **Categoria:** enumeração de categorias.
- **AppData:** estrutura agregadora (mapa de utilizadores).

0.4.4 Persistência (`data/`)

A camada `data/` é responsável por leitura e escrita em disco:

- **Persistencia:** serializa `AppData` para `appData/users.dat`.
- **PreferencesStore:** lê/escreve `appData/prefs.properties` (preferências globais pré-login).

0.4.5 Estrutura de pastas (resumo)

Pasta	Responsabilidade
<code>ui/</code>	Interface JavaFX, FXML, controladores, temas CSS e arranque da app
<code>ui/views/</code>	Ficheiros FXML das vistas
<code>ui/style/</code>	CSS base e modo escuro
<code>services/</code>	Lógica de aplicação e gestão de estado
<code>models/</code>	Entidades de domínio e estruturas de dados serializáveis
<code>data/</code>	Persistência local (dados e preferências)

0.5 Fluxos principais da aplicação

0.5.1 Arranque

No arranque, `App.start` lê as preferências globais (pré-login) e abre a vista de autenticação com tema e formatos já aplicados. Este detalhe melhora a consistência visual desde o primeiro ecrã e reduz mudanças abruptas após autenticação.

0.5.2 Registo e login

- **Registo:** o `RegisterController` cria um utilizador, inicializa preferências com o estado atual e navega para a vista principal.
- **Login:** o `LoginController` valida credenciais, carrega dados do utilizador e aplica as preferências específicas.

0.5.3 Gestão de hobbies

A listagem apresenta uma tabela com pesquisa textual e ações por linha. O diálogo `AddHobbyView` permite criar/editar, e a remoção é protegida por confirmação para reduzir apagamentos acidentais.

0.5.4 Gestão de sessões

O ecrã de sessões inclui tabela e pesquisa textual. O diálogo `AddSessionView` valida datas/horas e calcula automaticamente a duração em minutos, garantindo consistência e reduzindo erro humano.

0.5.5 Estatísticas e definições

- **Estatísticas:** o `StatsController` calcula agregados e atualiza gráficos e cartões de resumo.
- **Definições:** o `SettingsController` altera preferências e permite limpar dados do utilizador.

0.6 Modelo de dados

0.6.1 Entidade User

Um utilizador agrega:

- **Credenciais:** *username* e *password*.
- **Dados:** listas de **Hobby** e **Sessão**.
- **Preferências:** tema, formato de hora (12/24), formato de data e cor dos gráficos.

0.6.2 Entidade Hobby

Cada hobby inclui:

- Nome (identificação principal para o utilizador)
- Categoria (`Categoria`)
- Descrição (contexto e detalhes)

0.6.3 Entidade Sessão

Uma sessão associa-se a um hobby e armazena:

- Data e hora de início/fim (ou dados equivalentes)
- Duração em minutos (calculada automaticamente)
- Notas opcionais

0.6.4 Estrutura agregadora `AppData`

`AppData` mantém um mapa de utilizadores, com chave *case-insensitive*, simplificando a deteção de duplicados na criação de conta.

0.7 Persistência e salvaguarda

0.7.1 Dados principais

Os dados são persistidos por serialização de `AppData` (via *streams*) para o ficheiro:

`appData/users.dat`

A gravação valida a existência da pasta `appData/` (pré-requisito do projeto), reduzindo falhas por caminho inexistente.

0.7.2 Preferências globais (pré-login)

Para garantir uma experiência consistente logo no ecrã inicial, as preferências globais (como tema e formatos) são guardadas em:

`appData/prefs.properties`

0.7.3 Momento de gravação

A persistência é acionada automaticamente:

- após login/registo,
- após alterações em hobbies e sessões,
- ao alterar preferências.

Esta estratégia minimiza perda de dados em encerramentos inesperados e reduz a necessidade de ações manuais (*botão guardar*).

0.8 Interface e experiência de utilização

0.8.1 Estrutura de navegação

A vista principal (`MainView`) carrega sub-vistas dentro de um `StackPane` + `ScrollPane`, permitindo:

- alternar secções (hobbies, sessões, estatísticas, definições) de forma consistente;
- reutilizar a mesma moldura de interface;
- manter uma hierarquia de navegação simples.

0.8.2 Temas e estilos

Os estilos estão organizados em vários ficheiros CSS (base e modo escuro). O modo escuro é ativado por preferência, com aplicação imediata na interface.

0.8.3 Validações e *feedback*

Os diálogos e ecrãs incluem validações de input (campos obrigatórios, coerência temporal nas sessões) e mensagens de feedback, reduzindo erros e tornando o comportamento da aplicação mais previsível.

0.8.4 Gráficos

Os gráficos de barras alteram dinamicamente a cor consoante a preferência do utilizador, assegurando personalização e melhor legibilidade conforme o tema selecionado.

0.9 Justificação das opções técnicas

0.9.1 JavaFX + FXML + CSS

A opção por JavaFX com FXML separa a definição visual (FXML) da lógica de controlo (controladores), facilitando:

- manutenção do layout sem alterar código de lógica;

- *styling* via CSS e suporte simples a temas;
- reutilização de componentes e vistas.

0.9.2 Serialização local (*ObjectOutputStream*)

A persistência por serialização foi selecionada por ser suficiente num cenário académico e local:

- integração rápida e poucas dependências externas;
- desempenho adequado para o volume de dados esperado;
- simplicidade de implementação e de testes manuais.

0.9.3 AppState como ponto central

A utilização de um estado central (tipo *singleton*) permite:

- garantir um ponto único de verdade para o utilizador autenticado;
- reduzir duplicação de lógica de carregamento/gravação;
- facilitar a aplicação de preferências e a propagação de alterações.

0.9.4 Preferências aplicadas antes do login

Manter preferências globais pré-login melhora a UX ao:

- evitar mudanças de tema/formato imediatamente após autenticação;
- garantir consistência visual desde o primeiro contacto com a aplicação.

0.9.5 Hobbies sugeridos via enumeração

O uso de uma lista/enumeração de hobbies sugeridos acelera a aopção e reduz fricção inicial, além de promover categorias coerentes e repetíveis.

0.10 Limitações e melhorias futuras

Apesar de funcional, a solução apresenta limitações e oportunidades de evolução:

- **Remoção de conta:** sinalizada como não implementada (necessita abordagem segura).
- **Segurança:** passwords e ficheiros não são encriptados; adequado apenas para uso local académico.
- **Robustez da persistência:** serialização é menos resiliente a corrupção; evolução possível para SQLite ou JSON.
- **Validação avançada:** políticas de password forte e validações adicionais podem melhorar a fiabilidade.
- **Funcionalidades extra:** exportação/importação de dados, sincronização *cloud* e relatórios.

0.11 Como executar e testar

0.11.1 Requisitos

- JDK 17 (ou superior)
- JavaFX configurado no ambiente (SDK ou dependência na IDE)

0.11.2 Execução

Passos típicos:

1. Garantir a existência da pasta `appData/` na raiz do projeto (necessária para persistência).
2. Compilar o projeto (por exemplo, via IDE ou `Ant/build.xml`).
3. Executar a classe `ui.App`.

0.11.3 Testes manuais sugeridos

- Criar conta, iniciar sessão e confirmar persistência após reiniciar.
- Alternar tema e formatos (pré e pós-login), confirmando aplicação imediata.
- Criar/editar/apagar hobbies e confirmar pesquisa/filtragem.
- Criar/editar/apagar sessões e validar cálculo de duração e coerência temporal.
- Verificar estatísticas, gráficos e lista de recentes com dados variados.
- Usar a limpeza de dados do utilizador e confirmar efeito na interface e na persistência.

0.12 Conclusão

A aplicação *Gestor de Hobbies* cumpre o objetivo de permitir gestão de hobbies e registo de sessões, incluindo estatísticas e preferências personalizáveis com persistência local. A estrutura por camadas (UI/serviços/modelos/dados) facilita manutenção e evolução, e as opções técnicas tomadas privilegiam simplicidade, integração rápida e boa experiência de utilização. Como trabalho futuro, recomenda-se reforço de segurança, maior robustez de persistência e expansão funcional (exportação/importação e sincronização).